

```

"""
Excel Insight Dashboard
-----

Upload an Excel file and get an automatic, interactive dashboard with
descriptive statistics, distributions, correlations, and filtering.

Run with:
    streamlit run app.py
"""

import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

# -----
# Page config & style
# -----
st.set_page_config(
    page_title="Excel Insight Dashboard",
    page_icon="📊",
    layout="wide",
    initial_sidebar_state="expanded",
)

PRIMARY = "#6C63FF"
ACCENT = "#00C2A8"

st.markdown(
    f"""
<style>
    .main {{ background-color: #fafafa; }}
    .stMetric {{
        background-color: white;
        border: 1px solid #eee;
        border-radius: 12px;
        padding: 10px 15px;
        box-shadow: 0 1px 3px rgba(0,0,0,0.06);
    }}
    h1, h2, h3 {{ color: #2b2b3b; }}
    .block-container {{ padding-top: 2rem; }}
    div[data-testid="stMetricValue"] {{ color: {PRIMARY}; }}
</style>

```

```

        """
        unsafe_allow_html=True,
    )

px.defaults.template = "plotly_white"
px.defaults.color_continuous_scale = "Viridis"
COLOR_SEQ = px.colors.qualitative.Set2

# -----
# Helpers
# -----
@st.cache_data(show_spinner=False)
def load_excel(file):
    """Load all sheets from an uploaded Excel file."""
    xls = pd.ExcelFile(file)
    sheets = {name: xls.parse(name) for name in xls.sheet_names}
    return sheets

def clean_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    """Light auto-cleaning: drop empty cols/rows, try to parse dates."""
    df = df.dropna(axis=1, how="all").dropna(axis=0, how="all")
    df.columns = [str(c).strip() for c in df.columns]

    for col in df.columns:
        if df[col].dtype == object:
            # Try numeric conversion
            converted = pd.to_numeric(df[col], errors="coerce")
            if converted.notna().sum() >= 0.8 * df[col].notna().sum() and df[col]
                df[col] = converted
                continue
            # Try date conversion
            try:
                converted_dates = pd.to_datetime(df[col], errors="coerce")
                if converted_dates.notna().sum() >= 0.8 * df[col].notna().sum() a
                    df[col] = converted_dates
            except Exception:
                pass
    return df

def get_column_types(df: pd.DataFrame):
    numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
    datetime_cols = df.select_dtypes(include="datetime64[ns]").columns.tolist()
    categorical_cols = [
        c for c in df.columns

```

```

        if c not in numeric_cols and c not in datetime_cols
    ]
    # Treat low-cardinality numeric columns as potentially categorical too
    return numeric_cols, categorical_cols, datetime_cols

# -----
# Sidebar – upload & filters
# -----
st.sidebar.title("📊 Excel Insight Dashboard")
st.sidebar.write("Upload a spreadsheet to get started.")

uploaded_file = st.sidebar.file_uploader(
    "Upload Excel file", type=["xlsx", "xls", "xlsm"]
)

if uploaded_file is None:
    st.title("📊 Excel Insight Dashboard")
    st.markdown(
        """
        Welcome! This app turns any Excel file into an interactive,
        visual dashboard – automatically.

        **What it does:**
        - 📂 Reads every sheet in your workbook
        - 🪄 Auto-detects numeric, date, and categorical columns
        - 📈 Builds summary statistics, distributions, and correlations
        - 🔍 Lets you filter and explore the data interactively

        📁 Upload an `.xlsx` file using the sidebar to begin.
        """
    )
    st.info("No file uploaded yet. Try one of your own spreadsheets, or any export")
    st.stop()

# Load workbook
try:
    sheets = load_excel(uploaded_file)
except Exception as e:
    st.error(f"Couldn't read this file. Error: {e}")
    st.stop()

sheet_names = list(sheets.keys())
if len(sheet_names) > 1:
    selected_sheet = st.sidebar.selectbox("Select sheet", sheet_names)
else:
    selected_sheet = sheet_names[0]

```

```

raw_df = sheets[selected_sheet]
df = clean_dataframe(raw_df.copy())

if df.empty:
    st.warning("This sheet appears to be empty after cleaning.")
    st.stop()

numeric_cols, categorical_cols, datetime_cols = get_column_types(df)

# ---- Sidebar filters ----
st.sidebar.markdown("----")
st.sidebar.subheader("🔍 Filters")

filtered_df = df.copy()

with st.sidebar.expander("Categorical filters", expanded=False):
    for col in categorical_cols:
        unique_vals = df[col].dropna().unique().tolist()
        if 1 < len(unique_vals) <= 50:
            selected_vals = st.multiselect(f"{col}", sorted(map(str, unique_vals))
            if selected_vals:
                filtered_df = filtered_df[filtered_df[col].astype(str).isin(selected_vals)]

with st.sidebar.expander("Numeric range filters", expanded=False):
    for col in numeric_cols:
        col_min = float(df[col].min())
        col_max = float(df[col].max())
        if col_min < col_max:
            sel_range = st.slider(f"{col}", col_min, col_max, (col_min, col_max))
            filtered_df = filtered_df[
                (filtered_df[col] >= sel_range[0]) & (filtered_df[col] <= sel_range[1])
            ]

if datetime_cols:
    with st.sidebar.expander("Date filters", expanded=False):
        for col in datetime_cols:
            min_date = df[col].min()
            max_date = df[col].max()
            if pd.notna(min_date) and pd.notna(max_date) and min_date < max_date:
                date_range = st.date_input(f"{col}", (min_date.date(), max_date.date()))
                if isinstance(date_range, tuple) and len(date_range) == 2:
                    start, end = date_range
                    filtered_df = filtered_df[
                        (filtered_df[col] >= pd.Timestamp(start))
                        & (filtered_df[col] <= pd.Timestamp(end))
                    ]

```

```

st.sidebar.markdown("---")
st.sidebar.caption(f"Showing **{len(filtered_df):,}** of **{len(df):,}** rows")

# -----
# Main layout
# -----
st.title("📊 Excel Insight Dashboard")
st.caption(f"Sheet: **{selected_sheet}** · {df.shape[0]:,} rows × {df.shape[1]} c

tab_overview, tab_stats, tab_visuals, tab_correlation, tab_data = st.tabs(
    ["🏠 Overview", "📊 Descriptive Stats", "📈 Visual Explorer", "🔗 Correlatio
)

# ----- Overview tab -----
with tab_overview:
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Rows", f"{filtered_df.shape[0]:,}")
    c2.metric("Columns", f"{filtered_df.shape[1]:,}")
    c3.metric("Numeric columns", len(numeric_cols))
    c4.metric("Missing values", f"{int(filtered_df.isna().sum().sum()):,}")

    st.markdown("#### Column overview")
    overview_rows = []
    for col in filtered_df.columns:
        overview_rows.append({
            "Column": col,
            "Type": str(filtered_df[col].dtype),
            "Unique values": filtered_df[col].nunique(),
            "Missing": int(filtered_df[col].isna().sum()),
            "Missing %": round(100 * filtered_df[col].isna().mean(), 1),
        })
    st.dataframe(pd.DataFrame(overview_rows), use_container_width=True, hide_inde

    if numeric_cols:
        st.markdown("#### Quick look – first numeric column")
        first_num = numeric_cols[0]
        fig = px.histogram(
            filtered_df, x=first_num, nbins=30,
            color_discrete_sequence=[PRIMARY],
            title=f"Distribution of {first_num}",
        )
        st.plotly_chart(fig, use_container_width=True)

# ----- Descriptive stats tab -----
with tab_stats:
    st.markdown("#### Numeric summary")

```

```

if numeric_cols:
    desc = filtered_df[numeric_cols].describe().T
    desc["median"] = filtered_df[numeric_cols].median()
    desc["skew"] = filtered_df[numeric_cols].skew()
    desc["variance"] = filtered_df[numeric_cols].var()
    desc = desc[["count", "mean", "median", "std", "variance", "skew", "min",
    st.dataframe(desc.round(3), use_container_width=True)
else:
    st.info("No numeric columns detected in this sheet.")

if categorical_cols:
    st.markdown("#### Categorical summary")
    cat_col = st.selectbox("Choose a categorical column", categorical_cols, k
    value_counts = filtered_df[cat_col].astype(str).value_counts().reset_inde
    value_counts.columns = [cat_col, "Count"]
    col1, col2 = st.columns([1, 1])
    with col1:
        st.dataframe(value_counts, use_container_width=True, hide_index=True)
    with col2:
        top_n = value_counts.head(15)
        fig = px.bar(
            top_n, x="Count", y=cat_col, orientation="h",
            color="Count", color_continuous_scale="Viridis",
            title=f"Top categories - {cat_col}",
        )
        fig.update_layout(yaxis=dict(autorange="reversed"))
        st.plotly_chart(fig, use_container_width=True)

# ----- Visual explorer tab -----
with tab_visuals:
    st.markdown("#### Build your own chart")
    chart_type = st.selectbox(
        "Chart type",
        ["Histogram", "Box plot", "Scatter plot", "Bar chart", "Line chart", "Pie
    )

if chart_type == "Histogram" and numeric_cols:
    col = st.selectbox("Column", numeric_cols, key="hist_col")
    color_col = st.selectbox("Color by (optional)", ["None"] + categorical_co
    fig = px.histogram(
        filtered_df, x=col,
        color=None if color_col == "None" else color_col,
        nbins=40, color_discrete_sequence=COLOR_SEQ,
        marginal="box",
    )
    st.plotly_chart(fig, use_container_width=True)

```

```

elif chart_type == "Box plot" and numeric_cols:
    col = st.selectbox("Numeric column", numeric_cols, key="box_col")
    group_col = st.selectbox("Group by (optional)", ["None"] + categorical_co
    fig = px.box(
        filtered_df, y=col,
        x=None if group_col == "None" else group_col,
        color=None if group_col == "None" else group_col,
        color_discrete_sequence=COLOR_SEQ,
        points="outliers",
    )
    st.plotly_chart(fig, use_container_width=True)

elif chart_type == "Scatter plot" and len(numeric_cols) >= 2:
    x_col = st.selectbox("X axis", numeric_cols, key="scatter_x")
    y_col = st.selectbox("Y axis", [c for c in numeric_cols if c != x_col], k
    color_col = st.selectbox("Color by (optional)", ["None"] + categorical_co
    fig = px.scatter(
        filtered_df, x=x_col, y=y_col,
        color=None if color_col == "None" else color_col,
        color_discrete_sequence=COLOR_SEQ,
        trendline="ols" if st.checkbox("Show trendline") else None,
    )
    st.plotly_chart(fig, use_container_width=True)

elif chart_type == "Bar chart" and categorical_cols:
    cat_col = st.selectbox("Category column", categorical_cols, key="bar_cat"
    if numeric_cols:
        agg_col = st.selectbox("Value column", ["Count"] + numeric_cols, key=
        agg_func = st.selectbox("Aggregation", ["sum", "mean", "median", "max
    else:
        agg_col, agg_func = "Count", "sum"

    if agg_col == "Count":
        data = filtered_df[cat_col].astype(str).value_counts().reset_index()
        data.columns = [cat_col, "Count"]
        y_field = "Count"
    else:
        data = filtered_df.groupby(filtered_df[cat_col].astype(str))[agg_col]
        y_field = agg_col

    data = data.sort_values(y_field, ascending=False).head(25)
    fig = px.bar(data, x=cat_col, y=y_field, color=y_field, color_continuous_
    st.plotly_chart(fig, use_container_width=True)

elif chart_type == "Line chart" and datetime_cols and numeric_cols:
    date_col = st.selectbox("Date column", datetime_cols, key="line_date")
    value_col = st.selectbox("Value column", numeric_cols, key="line_val")

```

```

line_data = filtered_df.dropna(subset=[date_col]).sort_values(date_col)
fig = px.line(line_data, x=date_col, y=value_col, color_discrete_sequence
st.plotly_chart(fig, use_container_width=True)

elif chart_type == "Pie chart" and categorical_cols:
    cat_col = st.selectbox("Category column", categorical_cols, key="pie_cat")
    data = filtered_df[cat_col].astype(str).value_counts().reset_index().head
    data.columns = [cat_col, "Count"]
    fig = px.pie(data, names=cat_col, values="Count", color_discrete_sequence
st.plotly_chart(fig, use_container_width=True)

else:
    st.info("Not enough of the right column types are available for this char

# ----- Correlation tab -----
with tab_correlation:
    if len(numeric_cols) >= 2:
        st.markdown("#### Correlation matrix")
        corr = filtered_df[numeric_cols].corr()
        fig = px.imshow(
            corr, text_auto=".2f", color_continuous_scale="RdBu", zmin=-1, zmax=1
            aspect="auto",
        )
        st.plotly_chart(fig, use_container_width=True)

        st.markdown("#### Strongest relationships")
        corr_pairs = (
            corr.where(~np.eye(len(corr), dtype=bool))
            .stack()
            .reset_index()
        )
        corr_pairs.columns = ["Variable 1", "Variable 2", "Correlation"]
        corr_pairs["abs_corr"] = corr_pairs["Correlation"].abs()
        corr_pairs = corr_pairs.sort_values("abs_corr", ascending=False).drop_dup
        st.dataframe(corr_pairs.head(10)[["Variable 1", "Variable 2", "Correlatio
    else:
        st.info("Need at least two numeric columns to compute correlations.")

# ----- Raw data tab -----
with tab_data:
    st.markdown("#### Filtered data")
    st.dataframe(filtered_df, use_container_width=True)
    csv = filtered_df.to_csv(index=False).encode("utf-8")
    st.download_button("📄 Download filtered data as CSV", csv, "filtered_data.cs

```